PERFORMANCE REPORT

PDF OCR pipeline optimization

Summary

Check out https://nihal111.github.io/pdf-ocr-inference/ for a more interactive view

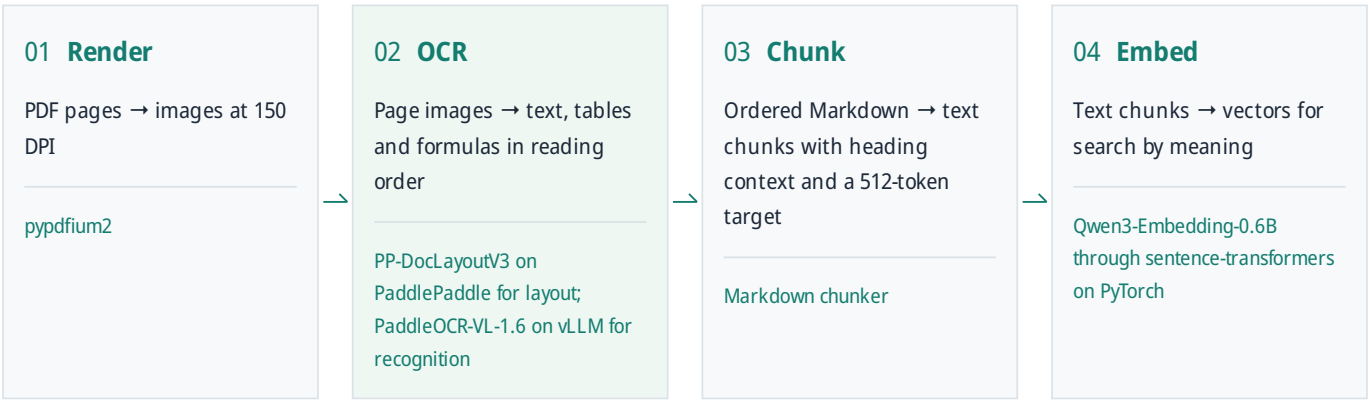
The problem: Turn PDFs into complete, ordered text and searchable embeddings faster, while preserving baseline OCR text and chunk structure and keeping embedding cosine similarity at or above 0.99.

Benchmark: Five frozen public PDFs, totaling 20 selected pages: *Earthquake Preparedness Checklist — English* (2), *The Estimation of Maximum Load Capacity of Seaplanes and Flying Boats* (3), IRS *Form W-9* (5), *Income in the United States: 2023* (5), and *Attention Is All You Need* (5).

System: Four NVIDIA L4 GPUs (22.5 GiB reported memory and a 72 W limit each); AMD EPYC 7R13, 96 logical CPUs, 363 GiB RAM.

Evaluation: Sequential PDF calls with one warm-up and five measured repeats per PDF (25 measured calls), checked against frozen baseline references.

Document pipeline



Results and scope

Eval metric	vLLM batch-1 baseline	Best optimized configuration	Speedup
Full-PDF p50	15.10 s	**2.34 s**	**6.44×**
Full-PDF p95	36.11 s	**12.41 s**	**2.91×**
Pages/minute	12.27	**45.23**	**3.69×**

Both variants were rerun on 2026-10-27 from clean revision `12f9cf6` and passed all 30 calls each against the existing references; no recapture was needed. Time-to-first-result was **41.40 s baseline / 131.34 s optimized**. This includes client startup and the first warm-up, but excludes recognizer-server startup. Fresh paired evaluation · Complete results and configurations.

Quality problems before optimization

Qwen3-VL-2B-Instruct produced reproducible but incomplete output. With a 2,048-token output limit, page transcriptions from the three-page report *The Estimation ofMaximum Load CapacityofSeaplanes and Flying Boats* were truncated. Matching passed because the references were also truncated. An 8,192-token retry confirmed repeated paragraphs; larger limits and tested penalties did not fix the setup. We switched to PaddleOCR-VL-1.6.

PaddleOCR-VL-1.6 repeated a table in IRS Form W-9. On the *Form W-9* benchmark, repetition penalties of 1.0–1.2 exhausted 8,192 tokens; 1.3 stopped but omitted most of the form. Raising the table-image limit from 1.0 to 2.8 million pixels resolved the tested crop; all 182 regions across the five selected pages reached the end-of-sequence marker (EOS). This setting is shared by the working baseline and all optimizations. Debugging evidence and external reports.

Repeatable vLLM output required batch invariance. Enabling it made repeated calls reproduce the same OCR text. The working baseline uses PaddleOCR-VL-1.6 on vLLM with one recognition region per GPU at a time; every optimization is checked against its frozen references. Baseline validation.

EOS means generation finished, not that it was accurate. References still contain errors in small table digits and merged cells. For example, on page 3 of *The Estimation ofMaximum Load CapacityofSeaplanes and Flying Boats*, Table V's first maximum-load value is **28,750** in the PDF, but the frozen OCR reference records **23,750**. Speedups preserve that reviewed baseline, not character-perfect transcription.

What GPU and system profiles showed

- **OCR was the first optimization target:** In the initial five-PDF evaluation, OCR consumed **89.9% of measured pipeline time** in the vLLM batch-1 baseline, excluding warm-ups. Each GPU read one region at a time, and regions were tied to the GPU handling their page. Increasing recognition batches to four reduced pooled full-PDF p50 from **15.08 to 9.55 seconds**; sharing regions across all four GPUs reduced it further to **5.39 seconds**, with embedding unchanged. OCR batching and distribution results.
- **Long table decoding controlled tail latency:** Before speculative decoding, a table region in the fifth selected page of *Income in the United States: 2023* generated 3,252 tokens. One GPU stayed about 98% busy while the others averaged 5–10%. Region batching could not divide that sequence's serial decoding, motivating the four-token speculative decoding experiment. Speculative decoding study.
- **Embedding became significant after the initial OCR gains:** With four-region batches shared across GPUs, *The Estimation ofMaximum Load CapacityofSeaplanes and Flying Boats* spent 1.89 seconds in embedding, or 35.5% of mean call time. This prompted the subsequent tests of lower embedding precision and smaller batches. Embedding profile.
- **Best optimized configuration:** When processing *The Estimation ofMaximum Load CapacityofSeaplanes and Flying Boats*, GPU activity was 68.7–82.3%, with only seven samples per GPU. Busy time is not achieved compute or bandwidth, so saturation remains unproven. GPU profile.

Changes that paid off

Each row compares a retained change with its recorded control, in experiment order.

Change	p50 gain	p95 gain	Pages/min gain
OCR regions per GPU: 1 → 4	**+57.9%**	+13.0%	+29.1%
Share regions across all GPUs instead of page-assigned GPUs	**+77.3%**	+5.0%	+26.3%
Embedding: FP32 → BF16; chunk batch 32 → 16	+28.6%	+7.4%	+12.4%
Region submission window: 16 → 64	+10.0%	+0.2%	+4.5%
Assign each page to the next free worker	−4.2%	+5.5%	+6.3%
Server admission: 4 → 8 requests per GPU	+20.4%	+4.1%	+5.7%
Compile embedding inference	+5.0%	+1.6%	+2.1%
Verify four n-gram draft tokens per decoder step	+3.0%	**+59.5%**	**+33.5%**
Dispatch regions when a GPU has capacity	+4.1%	−1.8%	+1.0%
Use fixed 32-row decoder matrix tiles	+13.7%	+22.0%	+18.4%
Embedding replicas: 1 → 4 GPUs	+7.0%	+3.4%	+3.9%
Render inside OCR workers, avoiding parent image transfers	+3.4%	+0.2%	+0.9%

Region batching and sharing produced the largest p50 gains. Speculative decoding produced the largest tail-latency and throughput gains. Dynamic page assignment initially traded a slower p50 for better p95 and throughput; raising admission to eight then improved all three metrics.

Hurdles and diminishing returns

Token-dependent tile selection initially changed only the vocabulary head: compilation froze the decoder's larger tiles. Fixed 32-row tiles reached the intended operations. Compilation also made cold starts expensive: an earlier 442-second suite included a 250-second first warm-up. The fresh time-to-first-result observations are reported above; the latency and throughput gains describe warmed-up workers.

Trial	What we learned
Sixteen requests per GPU	Slower p50, p95 and throughput than eight.
Draft depth and admission limits	Two drafts regressed; three/eight drafts and nearby admission limits did not establish a useful overall gain.
Per-projection decoder columns	Intended kernels ran, but full-PDF changes were under 1%.
Rendering formats and separate render pools	Compatible formats were near repeat variation; forced alpha changed pixels. Separate pools had mixed results and image-transfer costs.

Rendering/OCR now accounts for **97.1% of measured time** across the dataset. Eliminating embedding entirely would improve throughput by at most **2.5%**; eliminating preparation and chunking yields at most **0.5%**. These are conditional bounds, not measured speedups. Calculation details.

The final configuration passed the frozen-reference checks and delivered a **6.44× p50 speedup** on the five-PDF benchmark, using the same four L4 GPUs.

Theoretical limits

The optimized pipeline takes **26.53 s** to process all five PDFs one after another (20 pages, 136 chunk embeddings, warm models on four L4 GPUs). A best-case model gives **4.4–5.4 s** for the same serial workload, assuming perfect four-draft speculative acceptance, peak GPU bandwidth and negligible CPU and transfer overhead. It is an optimistic bound, not an achievable target.

Stage, all five PDFs serially	Measured	Optimistic minimum	Stage estimate
Rendering + OCR	25.76 s	4.29 s	5.34 s
Embedding	0.63 s	0.08 s	0.08 s
Preparation, chunking and other overhead	0.13 s	0, assumed	0, assumed
Total	**26.53 s**	**4.37 s**	**5.43 s**

- **Decoding is bandwidth-bound and sequential.** Each decoder pass reads about 721 MB of BF16 weights, which allows about 416 tokens/s per sequence. Four GPUs can decode different regions in parallel, but not one region's dependent tokens; the longest Census table needs 3,252 tokens.
- **Speculation sets the bound.** With perfect acceptance of four drafts, each pass advances five tokens. At two tokens per pass the bound rises to 7.7–9.3 s, and to 14–16 s with ordinary decoding.
- **OCR is where the gap is.** The workload is 24,890 generated tokens across 465 regions. Census (12.36 s measured vs 2.13 s ideal) and W-9 (7.68 s vs 1.68 s) account for most of the remaining distance.

Full calculation, assumptions and per-PDF breakdown

For more details, please open https://nihal111.github.io/pdf-ocr-inference/#/doc/results/summary